GPU Optimization Guide

A practical reference for training vision models without running out of memory

---

I put this together because every time I found a CUDA out of memory error, I spent an hour or two debugging the same handful of things. The fixes are almost always the same, and almost always take less than ten minutes once you know what to look for.

This guide is written for people who are new to training models on a GPU. It does **not** assume you have a background in systems or hardware. The companion memory calculator (see separate file) handles the specific numbers for each model.

Contents

1. Start here — the five things to do right now
2. What GPU memory actually is
3. Mixed-precision training (AMP)
4. Batch size, gradient accumulation, and DataLoader settings
5. More advanced techniques
6. Classification vs. segmentation
7. When things go wrong
8. Quick reference

---

1 Start Here

If you are having problems, try applying these five changes first. They resolve the vast majority of GPU problems, and you can come back later to understand the why behind each one.

1. Turn on mixed-precision training.

This is the single most impactful change you can make. It typically cuts memory by 35–45% and speeds training up 2–3x on any GPU made in the last five years. It takes about five extra lines of code and doesn't change your results.

```
from torch.cuda.amp import autocast, GradScaler
scaler = GradScaler()

for images, labels in train_loader:
    optimizer.zero_grad()
    with autocast():                    # that's it for the forward pass
        loss = criterion(model(images), labels)
    scaler.scale(loss).backward()       # scaler handles the rest
    scaler.step(optimizer)
    scaler.update()
```

2. Fix your DataLoader.

By default, PyTorch loads data on a single thread. That means your GPU finishes a batch and then sits idle while the CPU loads the next one. A few keyword arguments fix this:

```
import os
DataLoader(
    dataset,
    batch_size         = 32,
    num_workers        = min(os.cpu_count() // 2, 4),  # was 0
    pin_memory         = True,
    prefetch_factor    = 2,
    persistent_workers = True,
    drop_last          = True,   # prevents a smaller final batch causing OOM
)
```

3. Add torch.no_grad() to every validation loop.

model.eval() alone is not enough. Without no_grad(), PyTorch still builds the full computation graph during validation, wasting memory you can't afford. This is one of the most common oversights.

```
model.eval()
with torch.no_grad():
    with autocast():            # still worth using during inference
        outputs = model(images)
```

4. Use .item() when you log the loss.

If you append the raw loss tensor to a list, you're keeping its entire computation graph in memory. Every epoch, the list grows. This is why memory sometimes increases steadily over time even when nothing obvious changes.

```
losses.append(loss.item())   # correct
losses.append(loss)          # wrong — graph stays alive
```

5. Check memory before a long run.

Two lines tell you how much GPU memory you're actually using vs. how much you've peaked at. Run these after the first training step and you'll immediately see if something looks wrong.

```
print(round(torch.cuda.memory_allocated() / 1e9, 2), 'GB in use')
print(round(torch.cuda.max_memory_allocated() / 1e9, 2), 'GB peak')
```

Use the memory calculator to estimate whether a given model and batch size will fit before you start, especially useful for large segmentation models like SAM.

2 What Is GPU Memory?

Your graphics card has its own dedicated memory, called VRAM (or GPU memory), that's completely separate from the RAM in your computer. Everything involved in a training step has to fit inside this memory at once. When it doesn't fit, you get a CUDA out of memory error and the whole run crashes.

Four things consume VRAM during training:

- **Model parameters.** The actual weights of the network. Every number the model has learned. Fixed in size once you've picked an architecture.
- **Activations.** Intermediate results saved during the forward pass so the backward pass can compute gradients from them. This is almost always the largest cost, and it scales with batch size and image resolution. It's the main target for optimization.
- **Gradients.** One gradient value per trainable parameter. Same size as the parameters themselves. Zero for any layer you've frozen.
- **Optimizer state.** Extra data the optimizer needs to do its job. Adam, the most common optimizer, stores two extra tensors per parameter (running averages of past gradients). This can be surprisingly large.

The thing that catches people off guard is that activations aren't proportional to the number of parameters. They're proportional to the batch size times the resolution squared. That means a model with 25M parameters (like ResNet-50) can easily use ten times more activation memory than parameter memory when you're training on 256×256 images with a batch of 32.

Resolution is quadratic

This is the most common surprise for people new to training vision models. If you double the image dimensions, activation memory doesn't double, it quadruples. A 512×512 image requires 16x more activation memory than a 128×128 one.

Resolution doubles	`M_acts → 4×` *Quadratic scaling. The most expensive thing to change.*
Batch size halves	`M_acts → ÷ 2` *Linear. Safe first response to an OOM error.*

In practice: if you're hitting OOM, try halving the batch size first. If that doesn't help, try a smaller resolution. Only after both of those should you reach for more complex techniques.

Resolution	Memory (relative to 128px)	Max batch on 8GB GPU (AMP on)
128 × 128	1x — baseline	128–256
224 × 224	3.1x	32–64
256 × 256	4x	32
384 × 384	9x	8–16
512 × 512	16x	2–4
1024 × 1024 (SAM)	64x	1 — often inference-only

These are rough estimates for a mid-sized model with AMP enabled. Use the calculator for specific architectures.

The rough memory budget

Before launching an experiment, it's worth doing a quick estimate. Exact numbers aren't necessary. You're just checking whether something will obviously not fit.

Parameters (FP32)	`N × 4 bytes` *N = total parameter count*

Parameters (AMP)	`N × 6 bytes` *FP16 working copy (×2) + FP32 master (×4)*
Activations (approx)	`B × (H×W ÷ sf) × C × L × 2B × 1.5` *sf: CNN ≈ 20, Hybrid ≈ 10, Transformer ≈ 50*
Gradients	`N_trainable × 4B (FP32) or × 2B (AMP)` *Zero for frozen layers*
Adam states	`N_trainable × 8 bytes` *Switch to Adam 8-bit if this is large*

The sf value (spatial factor) accounts for the fact that CNNs progressively downsample — ResNet-50 shrinks from 224×224 down to 7×7 over 50 layers, so the average spatial size across all layers is much smaller than the input. Transformers work differently: they turn the image into a sequence of patches, so the 'spatial' size is fixed at $(H/patch_size)^2$. The calculator handles all of this automatically.

3 Mixed-Precision Training (AMP)

Modern GPUs have special hardware, Tensor Cores, that can do matrix multiplication in 16-bit floating point (FP16) about 2–3x faster than in 32-bit (FP32). AMP takes advantage of this automatically, while keeping 32-bit precision for the parts of training where it matters most.

The result: activation memory is halved (since each value now takes 2 bytes instead of 4), and most training steps run faster. Accuracy is essentially unchanged. PyTorch handles the precision transitions so you don't have to think about them.

What changes with AMP	FP32	AMP
Activation memory	Baseline	~50% reduction
Total VRAM	Baseline	~55–65% of FP32
Max batch size	Baseline	~2x larger possible
Training throughput	Baseline	1.5–3x faster
Accuracy	Baseline	Same in practice

Why there's a GradScaler

FP16 can't represent very small numbers. Anything below about 6×10^{-8} just becomes zero. Gradients in deep networks are often in this range, so without a workaround they'd silently vanish. The GradScaler multiplies the loss by a large number before backprop (keeping gradients in FP16's representable range), then divides back out before the optimizer step. It adjusts this scale factor automatically during training, so you never need to tune it.

> If you see NaN losses with AMP, don't immediately blame the precision. Check your learning rate first. AMP-induced NaNs from gradient underflow are less common than people expect, and are usually caught and skipped automatically by the scaler.

Optimizer memory

The optimizer is sometimes the second-biggest memory consumer, and it's easy to overlook. Adam stores two extra tensors per parameter (the first and second moment estimates), both in FP32 regardless of whether you're using AMP. For a model with 300M parameters, that's about 2.4GB just for optimizer state.

The options, in rough order of memory use:

Optimizer	Extra per param	Relative total	When to use it
SGD (no momentum)	nothing	1x	Rarely — too slow for most deep networks
SGD + momentum	1 FP32 tensor (4B)	2x	Simple fine-tuning, or when memory is very tight
Adam / AdamW	2 FP32 tensors (8B)	3x	The standard choice for most work
Adam 8-bit	2 INT8 tensors (2B)	~1.25x	Drop-in replacement for Adam, ~75% less optimizer memory
Adafactor	factored matrix (~0.4B)	~1.1x	Very large models (1B+ params), or extreme memory pressure

```
# Adam 8-bit: install with  pip install bitsandbytes
import bitsandbytes as bnb
optimizer = bnb.optim.AdamW8bit(model.parameters(), lr=1e-4)
# that's literally all you change
```

4 Batch Size, Accumulation, and DataLoader

Gradient accumulation

When your desired batch size doesn't fit in memory, don't just use a tiny batch and hope for the best. Gradient accumulation lets you run several small batches before taking a single optimizer step. So the model learns from the combined gradient of all of them, as if you had a larger batch, while the GPU only ever holds one small batch at a time.

Effective batch	`B_effective = B_micro × K` *K = steps between optimizer updates*
Memory cost	`cost of B_micro — not B_effective` *GPU never sees the full effective batch at once*
Learning rate	`scale LR by (B_effective / original B)` *keep the training dynamics equivalent*

```
K = 8                          # pretend we have 8x the batch size
optimizer.zero_grad()

for step, (images, labels) in enumerate(loader):
    with autocast():
```

```
        loss = criterion(model(images), labels) / K   # divide — don't skip
this
    scaler.scale(loss).backward()
    if (step + 1) % K == 0:
        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad()
```

> The division by K is not optional. Without it, the gradients accumulate to K times the correct magnitude and training usually diverges.

DataLoader settings

The GPU can't train faster than the CPU can load data. With num_workers=0 (the PyTorch default), the GPU finishes a batch and then sits idle while the CPU loads the next one. On a machine with a decent number of CPU cores, fixing this is often as impactful as any memory optimization.

Setting	Default	Better value	What it does
num_workers	0	2–4 per GPU	Parallel data loading — GPU stops waiting
pin_memory	False	True	~30% faster CPU-to-GPU transfer
prefetch_factor	2	2–4	Queues N batches per worker in advance
persistent_workers	False	True	Workers survive between epochs
drop_last	False	True (training)	Prevents smaller final batch causing OOM

A note on pre-loading batches to GPU

For maximum GPU utilization, it's worth overlapping data transfer with computation entirely. The idea: while the GPU is processing batch N, the next batch (N+1) is being transferred from CPU to GPU on a separate CUDA stream. By the time the GPU is ready for it, it's already there.

```
class CUDAPrefetcher:
    def __init__(self, loader):
        self.loader = iter(loader)
        self.stream = torch.cuda.Stream()   # separate stream for transfers
        self._preload()

    def _preload(self):
        try:   self.next = next(self.loader)
        except StopIteration: self.next = None; return
        with torch.cuda.stream(self.stream):     # transfer while GPU trains
            self.next = [t.cuda(non_blocking=True) for t in self.next]

    def __next__(self):
        torch.cuda.current_stream().wait_stream(self.stream)
```

```
        batch = self.next
        self._preload()          # kick off the next transfer immediately
        return batch

    def __iter__(self): return self

for images, labels in CUDAPrefetcher(train_loader):
    ...
```

Memory cost: one extra batch held on GPU at all times. At B=32, 256×256, RGB with AMP, that's roughly 25MB, not worth worrying about.

5 More Advanced Techniques

The things in this section are worth knowing about, but they're not step one. Get AMP and the DataLoader working first.

Gradient checkpointing

During the forward pass, PyTorch saves every intermediate activation so it can use them during backprop. Gradient checkpointing tells it to discard most of these and recompute them as needed during the backward pass instead. You pay ~33% more in compute, but activation memory drops by 40–60%. It's the right tradeoff when you're memory-bound but not compute-bound, which is most of the time on consumer GPUs.

Memory saving	`M_acts ÷ √L  (approximate)` *L = number of layers. More layers = bigger benefit.*
Compute cost	`training time × ~1.33` *Worth it in most cases when memory is the bottleneck*

```
# HuggingFace models — one line
model.gradient_checkpointing_enable()

# Custom modules
from torch.utils.checkpoint import checkpoint
x = checkpoint(self.layer, x)   # activations recomputed in backward
```

Frozen backbone

When fine-tuning a pretrained model, which is the standard approach for most vision tasks — you usually don't need to update every layer right away. Freezing the backbone means gradients aren't computed for those layers, so their activations don't need to be stored. This can halve or more the active memory cost.

```
for name, param in model.named_parameters():
    if 'classifier' not in name and 'head' not in name:
        param.requires_grad = False
```

```
# quick sanity check
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total     = sum(p.numel() for p in model.parameters())
print(f'Training {trainable/1e6:.1f}M of {total/1e6:.1f}M parameters')
```

A common schedule is to freeze everything for the first few epochs, then progressively unfreeze deeper layers. The backbone learning rate is usually 5–10x lower than the head to avoid destroying pretrained features.

Phase	Trainable layers	Memory	When
Head only	Final layer(s) only	Minimal	Epochs 1–3
Top blocks + head	Head + last 2–3 blocks	Moderate	Epochs 4–6
Full model	Everything (low LR backbone)	Full	Epoch 7+

torch.compile() — free performance on PyTorch 2.0+

If you're on PyTorch 2.0 or later, this is an easy win. It compiles your model's computation graph ahead of time, eliminating repeated overhead and enabling kernel fusion. The first forward pass is slower (it's compiling), but everything after is faster — sometimes significantly so.

```
model = torch.compile(model)
# first forward: slow. everything after: 20-100% faster.
# skip this during debugging — it makes tracebacks harder to read
```

6 Classification vs. Segmentation

Classification — the straightforward case

Classification models output a single label per image. They're the most memory-efficient vision task because the model's output is tiny (one number per class) regardless of image size. The techniques in sections 0–4 are generally sufficient for classification work.

Things worth knowing specifically for classification:

- **Resolution matters more than you'd expect.** For something like CLEAR-100 (30 classes, natural images), there's usually not much accuracy difference between 224px and 256px — but the memory difference is real. Start at 224.
- **Frozen backbone fine-tuning works very well** on smaller datasets like Yearbook or CLEAR-10. Full fine-tuning often overfits unless you have aggressive augmentation.
- **For binary classification,** the activation memory is identical to multi-class — only the final linear layer changes size. So going from 2 to 30 classes has almost no memory impact.

Segmentation — considerably more expensive

Segmentation models have to produce a class label for every single pixel in the output image. The output tensor alone is (batch × height × width × num_classes) — for a 512×512 image with 20

classes, that's 20MB per image just for the output, before any activations. On top of that, many segmentation architectures (U-Net and variants) hold onto all encoder feature maps to use as skip connections for the decoder, which can triple or quadruple memory vs. a comparable classification model.

Patch-based training

For large images — medical scans, satellite imagery, anything above about 512px — the standard approach is to train on random crops of a fixed size. Memory is then bounded by the crop size regardless of how large the original images are.

```
transforms.RandomCrop(512)    # memory is now always B × 512 × 512 × ...

# for inference on full images, stitch predictions with a sliding window:
def predict_full(model, image, patch=512, overlap=0.25):
    stride = int(patch * (1 - overlap))
    preds  = torch.zeros(1, num_classes, *image.shape[-2:]).cuda()
    counts = torch.zeros(1, 1,           *image.shape[-2:]).cuda()
    for y in range(0, image.shape[-2]-patch+1, stride):
        for x in range(0, image.shape[-1]-patch+1, stride):
            with torch.no_grad(), autocast():
                p = model(image[:,:,y:y+patch,x:x+patch])
            preds [:,:,y:y+patch,x:x+patch] += p
            counts[:,:,y:y+patch,x:x+patch] += 1
    return preds / counts
```

SAM and SAM2

SAM (Segment Anything Model) and its successor SAM2 are foundation models for segmentation. They operate at 1024×1024 resolution, which means even a single forward pass uses several gigabytes. A few things to know:

- **SAM ViT-H at batch=1 uses ~10–12GB** with AMP — inference-only for most consumer GPUs. Fine-tuning it requires freezing the image encoder and only training the lightweight prompt encoder and mask decoder (~4M of the 636M parameters).
- **SAM2 uses a Hiera backbone** (hierarchical ViT that downsamples like a CNN), which makes it substantially more memory-efficient than SAM ViT at the same resolution. SAM2-Tiny at batch=1 is in the same range as SAM ViT-B.
- **Always use torch.no_grad() for SAM inference.** It's easy to forget and the cost is enormous at 1024×1024.

The memory calculator includes presets for SAM ViT-B/L/H and all four SAM2 variants. Use it to check whether your GPU can handle inference at batch=1 before anything else.

7 When Things Go Wrong

OOM errors are rarely mysterious once you've seen a few of them. There are really only about six things that cause them, and each one has an obvious fix.

What happened	Likely cause	Fix
OOM on the first batch	Batch size or resolution is too large for the model	Enable AMP first. If still OOM, halve the batch size.
OOM partway through an epoch	Variable-size last batch, or images have different sizes	Add drop_last=True to your DataLoader.
Memory climbs with every epoch	Storing raw loss tensors — graph never freed	Replace loss_history.append(loss) with .append(loss.item()).
OOM during validation only	Missing torch.no_grad() in the eval loop	Wrap all validation code in with torch.no_grad():.
OOM after resolution increase	Activation memory increased by (new/old)²	Reduce batch size by the same factor, or enable gradient checkpointing.
OOM with multiple GPUs	DataParallel: master GPU holds extra model copies	Switch to DistributedDataParallel.

The order to try things

When you're hitting OOM and need to reduce memory, this is the order I'd go in. Each step is safer and easier than the next, so start at the top.

#	Action	Mem saved	Speed	Risk
1	Enable AMP (autocast + GradScaler)	35-45%	+50-200%	None
2	Fix DataLoader (num_workers, pin_memory, drop_last)	0%	Removes GPU idle time	None
3	Halve batch size	~40%	Minor throughput drop	None
4	Gradient accumulation to keep effective batch size	~0%	<1% overhead	None
5	Reduce resolution by 25% (saves ~44% activation memory)	~44%	Faster	Check accuracy
6	Gradient checkpointing	40-60%	-33% slower	None
7	Freeze backbone	60-80%	Much faster	Watch convergence
8	Adam to Adam 8-bit	~25% (optim only)	No change	None
9	torch.compile()	0%	+20-100%	PyTorch 2.0+ only

8 Quick Reference

Formulas and model numbers in one place, for when you need to check something quickly.

Memory formulas

Params (FP32)	`N × 4 bytes` *Fixed — doesn't change with batch or resolution*
Params (AMP)	`N × 6 bytes` *FP16 working copy + FP32 master*
Activations	`B × (H×W ÷ sf) × C_avg × L × 2B × 1.5` *sf: CNN=20, Hybrid=10, Transformer=50*
Resolution scales	`M_acts × (new_H / old_H)²` *Double the resolution, quadruple the activations*
Gradient accum	`memory = M(B_micro) not M(B_effective)` *B_micro × K = B_effective*
Checkpoint saving	`M_acts ÷ √L (approx)` *At the cost of 33% more compute*
Adam states	`N_trainable × 8 bytes` *Two FP32 running averages per parameter*
Patch training	`M_full × (crop ÷ H)²` *Bounds segmentation memory to the crop size*

Model reference

Model	Type	Params	Batch=1 AMP	Batch=32 AMP
MobileViT-XXS	Hybrid	1.3M	< 1GB	~3GB
MobileViT-S	Hybrid	5.6M	< 1GB	~4GB
EfficientNet-B1	CNN	7.8M	< 1GB	~4GB
ResNet-50	CNN	25.6M	< 1GB	~5GB
Swin-T	Hybrid	28M	~1GB	~6GB
ViT-Base/16	Transformer	86M	~1GB	~5GB
SAM ViT-B	Transformer	89M	~3GB†	> 10GB
SAM2-Tiny	Hybrid	38.9M	~2GB†	~7GB
SAM2-Large	Hybrid	224M	~5GB†	> 24GB
SAM ViT-H	Transformer	636M	~12GB†	> 80GB
DINOv2 ViT-L	Transformer	307M	~4GB	~16GB
CLIP ViT-L/14	Transformer	307M	~3GB	~14GB

† SAM variants use 1024×1024 input resolution. Memory is high even at batch=1.

Quick decision guide

If you need to...	Use this
Reduce memory immediately, no accuracy risk	AMP (Section 0, step 1)
Keep a large effective batch size on a small GPU	Gradient accumulation (Section 3)
Speed up data loading	Fix the DataLoader settings (Section 3)
Fine-tune a large pretrained model efficiently	Frozen backbone schedule (Section 4)
Train a segmentation model on high-res images	RandomCrop training + sliding window inference (Section 5)
Push even further on memory	Gradient checkpointing + Adam 8-bit (Section 4, Section 2)
Memory grows with every epoch	Tensor leak — check for loss.item() (Section 0, step 4)